

PRAKTIKUM INFORMATIK 2

M. Farmbauer, M. Niemetz

Objektbasiertes Softwaredesign

Digitalschaltung

Juni 2026, Version: 3.61

Vorbereitung

- ☐ ☕☕ **Containerklasse `std::vector` aus der C++ Standardbibliothek:** Versuchen Sie sich mit den Möglichkeiten der als Template ausgeführten Containerklasse vertraut zu machen. Einige Hinweise finden Sie im Skript aber die ausführlichere Information finden Sie in der C++ Dokumentation (gutes Buch oder <https://cplusplus.com/reference/vector/vector/>). 6
- ☐ ☕☕ **„rein virtuelle“ Funktionen und abstrakte Klassen:** Schlagen Sie nach, was „rein virtuelle“ Funktionen („pure functions“) sind, wie man sie erzeugt und was dies für die Klasse, die sie enthält und dadurch zu einer sogenannten abstrakten Klasse wird, bedeutet. 18

1 Überblick

In dieser Aufgabe können Sie das in diesem Semester Gelernte vertiefen und in einem komplexen Beispiel anwenden. Damit dient diese Aufgabe in besonderem Maß der Prüfungsvorbereitung, weshalb es sich lohnt, die Bearbeitung dieser Aufgabe mit besonderer Tiefe anzugehen. Nicht alle spannenden Details drängen sich bei oberflächlicher Bearbeitung nach vorne.

Inhaltlich wird ein Baukasten zur Simulation digitaler Logikschaltungen entwickelt. Dieser gestattet es, digitale Logikschaltungen aus verschiedenen grundlegenden Elementen der Digitaltechnik (z.B. OR-, AND- und NAND-Gatter) aufzubauen und zu simulieren.

Die Eingänge der Gatter können dabei mit Hilfe von `Switch`-Objekten auf Basis der Klasse `Switch` aus der vorangehenden Aufgabe interaktiv mit Eingangspegeln bedient werden. Die Zustände der Ausgänge werden über farbige Indikatoren visualisiert.

Abbildung 1 zeigt ein Beispiel für eine simulierte Logikschaltung.

Aufgrund der Vorarbeiten in vorangehenden Aufgaben, die Zerlegung in unterschiedliche Klassen und das schrittweise Vorgehen bei der Implementierung bleibt der Aufwand für die Implementierung zur Realisierung der in Abbildung 1 gezeigten Funktionalität überraschend gering.

Abbildung 2 zeigt die in dieser Aufgabe entwickelte Klassenfamilie sowie ihre Interaktionen anhand eines vereinfachten UML Klassendiagramms. Dabei kommen auch die in der vorangehenden Aufgabe entwickelten Klassen zum Einsatz. In diesem Diagramm wurden um der Übersichtlichkeit willen die Details weggelassen. So fehlen zum Beispiel die Klassen `Point` und `RGBColor` aber auch alle Datenelemente und Funktionen. Letztere finden sich dann bei den jeweiligen Teilaufgaben.

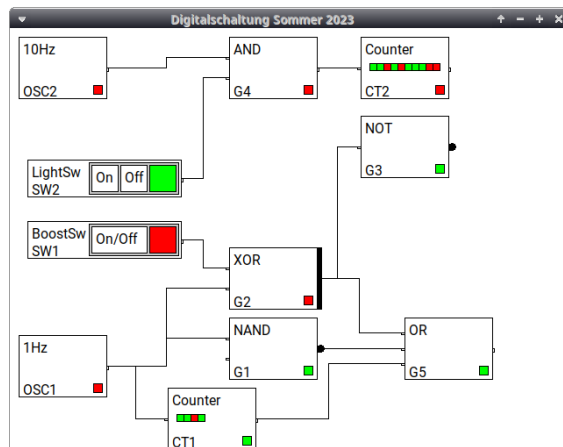


Abbildung 1: Beispiel für eine komplette Digitalschaltung unter Verwendung einiger der implementierten Komponenten.

Hinweis: Diese Aufgabe ist die letzte und mit Abstand komplexeste Pflichtaufgabe. Hier sollen Sie die abschließenden Themen einüben und eigenständig die nötigen Lösungen erarbeiten.

Nehmen Sie keinesfalls fremde Lösungen zu Hilfe, sondern erarbeiten Sie hier die Lösungen unbedingt selbstständig mit Hilfe von Büchern, Skript und ähnlichen Quellen. Sollten Sie dabei Schwierigkeiten haben, dann beseitigen Sie diese – gerne auch im Gespräch mit Ihren Dozenten in Vorlesung, Praktikum und Tutorium. Nur so können Sie in der Prüfung erfolgreich sein!

Wichtig: Diese Beschreibung ist sehr umfangreich in der Hoffnung, dass Sie all die Information enthält, die Sie vielleicht benötigen. Sie müssen dabei nicht alles „lesen und abspeichern“ um weitermachen zu können. Viel wichtiger ist es, sich zunächst einen Überblick zu verschaffen und zu verstehen, wie die einzelnen Klassen zusammenarbeiten. Die Details kann man nachlesen, wenn man sie benötigt. Lesen Sie also „nichtlinear“ die Dinge, die Sie gerade brauchen.

Diese Vorgehensweise ist **auch in der Prüfung** sehr hilfreich, denn auch dort müssen Sie in knapper Zeit den Überblick über eine Reihe von Klassen und deren Zusammenarbeit gewinnen.

2 Vorbereitung



Aktualisieren Sie ihren Fork des PIN2-Projekts, indem Sie `git pull upstream master` aufrufen. Überprüfen Sie anschließend, ob sich das Verzeichnis für diese Aufgabe nun in Ihrem Repository befindet.

Da zur Implementierung der notwendigen neuen Klassen die in der letzten Übungsaufgabe entwickelten Klassen verwendet werden, müssen Sie die entsprechenden `.h` und `.cpp` Dateien aus der letzten Aufgabe in das neue Projektverzeichnis kopieren und im CodeBlocks-Projekt integrieren.



Aktualisieren Sie Ihren Fork des PIN2-Projekts.



Integrieren Sie die in der Aufgabe entwickelten Klassen in Ihr neues Projekt.

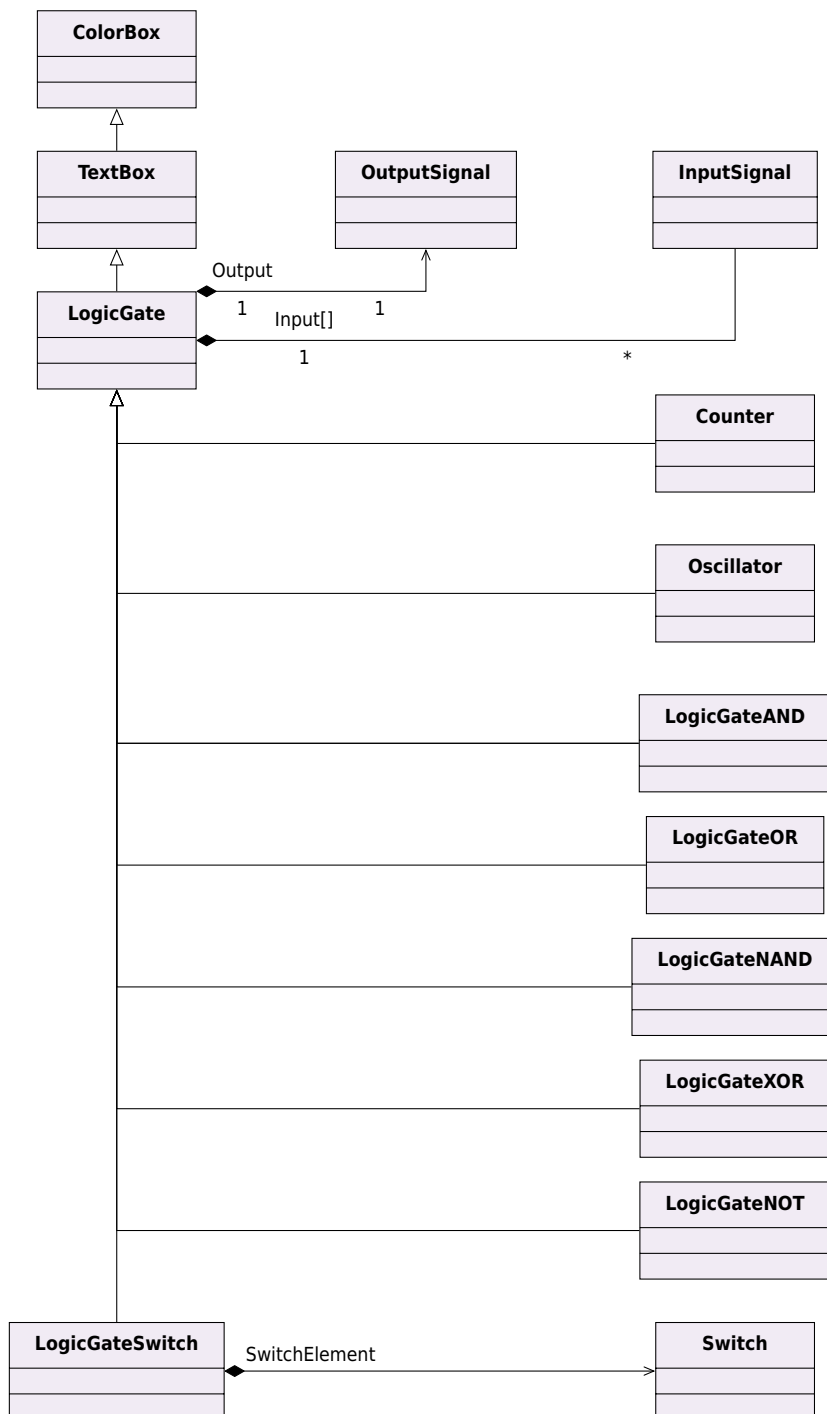


Abbildung 2: Das Projekt im Endausbau. Die verschiedenen Klassen bzw. Methoden werden im Verlauf der Aufgabe schrittweise implementiert. Die Klassen `Point` und `RGBColor` aus der vorangehenden Aufgabe wurden der Übersichtlichkeit halber weggelassen, die übrigen Klassen verkürzt dargestellt. Die Details folgen dann im weiteren Verlauf der Aufgabe.

Wichtig: Überschreiben Sie keine Dateien des Programmrahmens mit Ihren eigenen Versionen. Fügen Sie nur die fehlenden Dateien hinzu!

Die betrifft die Dateien mit den Klassen:

- `Point` (bereits mit erweiterter Version im Programmrahmen enthalten)
- `RGBColor` (bereits im Programmrahmen enthalten)
- `TextBox`
- `ColorBox`
- `Switch`
- `SwitchToggle`
- `SwitchOnOff`
- `SwitchToggle`

Hinweis: Vergessen Sie nicht, das CodeBlocks-Projekt nach der Integration der neuen Dateien zu speichern, bevor Sie den Zustand in einem Commit ablegen. Im Gegensatz zu den Source-Dateien wird das Projekt beim Compilieren nicht automatisch in die `.cbp`-Datei gesichert. Dafür speichern Sie entweder das Projekt oder schließen es (mit speichern) und öffnen es erneut.

Prüfen Sie nun, ob Ihr Projekt ordnungsgemäß funktioniert (es enthält einfachen Testcode). Beheben Sie ggf. vorhandene Probleme bevor Sie fortfahren.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

3 Beschreibung der Klasse LogicGate

Wichtig: Verschaffen Sie sich zunächst einen Überblick über das Klassendesign indem Sie die Beschreibung lesen und ggf. durch zusätzliche Zeichnungen für Klarheit sorgen, bevor Sie versuchen mit den Arbeitsanweisungen zu beginnen.

LogicGate
Input :vector<InputSignal> – Output :OutputSignal – Indicator :ColorBox – ID :string
+ LogicGate(Operation :string&, Position :Point&, ID :string&, NumInputs :unsigned) + ~LogicGate() + operator==(rhs :LogicGate&) :bool + getID() :string& + setPosition(Position :Point&) :void + setSize(Size :Point&) :void + show() :void + updateOutput() :void + getOutput(Port :unsigned) :bool + getNumOutputs() :unsigned + getNumInputs() :unsigned + connectOutput(Peer :LogicGate&, Port :unsigned) :void + connectInput(From :LogicGate&, CurrentState :bool Port :unsigned) :InputSignal& # decorate() :void # setOutput(NewState :bool, Port : unsigned) :void – updateIndicator() :void – positionElements() :void

Die Klasse **LogicGate** bildet den Kern der gesamten Simulationsumgebung und ist daher etwas umfangreicher.

Um Ihnen unnötige Tipparbeit zu ersparen, enthält der Programmrahmen bereits eine vollständige Klassendefinition. Allerdings sind nur manche der Funktionen der Klasse bereits in der .cpp-Datei implementiert.

Abbildung 4 zeigt die Klasse mit Ihrer Ableitungshistorie. Die bereits aus der letzten Aufgabe bekannten Klassen **TextBox** und deren Basisklasse **ColorBox** bilden die Basis.

Darauf aufbauend wird durch eine Ableitung ein generisches logisches Gatter **LogicGate** entwickelt, welches eine skalierbare Anzahl an Eingängen, abgebildet mit Objekten der Klasse **InputSignal**, sowie einen Ausgang, abgebildet über ein Objekt der Klasse **OutputSignal**, besitzt. Die Skalierbarkeit der Anzahl an Eingängen wird durch die Verwendung des Klassentemplates **std::vector** aus der Standardbibliothek realisiert, welche eine Array-artige Containerklasse darstellt, die beliebige Datentypen unterstützt und in unserem Fall Objekte des Typs **InputSignal** enthält.

Diese Objekte der Klassen **InputSignal** und **OutputSignal** organisieren die Kommunikation zwischen verschiedenen

Objekten der Klasse **LogicGate**. Dabei nehmen die Objekte von **InputSignal** die Signale der anderen Gatter entgegen, speichern Sie ab und informieren das **LogicGate**-Objekt, zu dem sie gehören, über die Änderung. Zudem stellen sie sicher, dass immer nur ein Ausgang mit dem jeweiligen Eingang verbunden ist.

Recherche ☕☕

Containerklasse **std::vector** aus der C++ Standardbibliothek

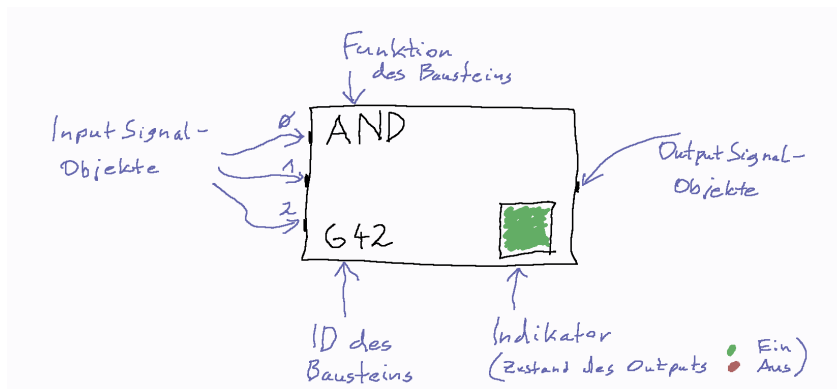


Abbildung 3: Entwurf eines Gatters (hier am Beispiel eines AND-Gatters mit drei Eingängen) samt der Anschlusspositionen für die Ein- und Ausgangssignale. Die in dieser Aufgabe entwickelten Klassen unterstützen alle lediglich ein Ausgangssignal aber eine Erweiterung ist in den Grundzügen angelegt.

Demgegenüber stellen die Objekte des Typs `OutputSignal` die Verbindungen des Ausgangs zu den Eingängen her und verwalten die bereits bestehenden Verbindungen unter Verwendung von `std::vector` in einer Liste, um im Bedarfsfall alle betroffenen Eingänge von einer Zustandsänderung des Ausgangs informieren zu können.

Jedes Objekt der Klasse `LogicGate` (bzw. der davon abgeleiteten Klassen) hat außerdem die Aufgabe, seine Eingangssignale durch eine geeignete Implementierung der Schaltlogik mit seinem Ausgang zu verbinden und damit verschiedene Logikgatter aber auch z.B. Counter zu implementieren. Außerdem ist natürlich eine Darstellung auf dem Grafikbildschirm bereit zu stellen.

Beginnen Sie damit, in der Klassendefinition der Klasse `LogicGate` die fehlenden `const`-Spezifikationen für Parameter, Rückgabewerte und Funktionen zu ergänzen. Falls Sie sich dabei nicht sicher sind, können Sie die Beschreibung der Datenelemente und Funktionen in den beiden nächsten Unterabschnitten zu Rate ziehen.

Wenn Sie dies erledigt haben, können Sie in der Datei `testlevel.h` den `TEST_LEVEL` auf den Wert `TEST_LOGICGATE` setzen und die Ausgabe des Compilers prüfen. Wegen der starken Verflechtung von `LogicGate`, `InputSignal` und `OutputSignal` gibt es aber noch keine Ausgabe auf dem Bildschirm.



Ergänzen Sie in `LogicGate` die fehlenden `const`-Spezifikationen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

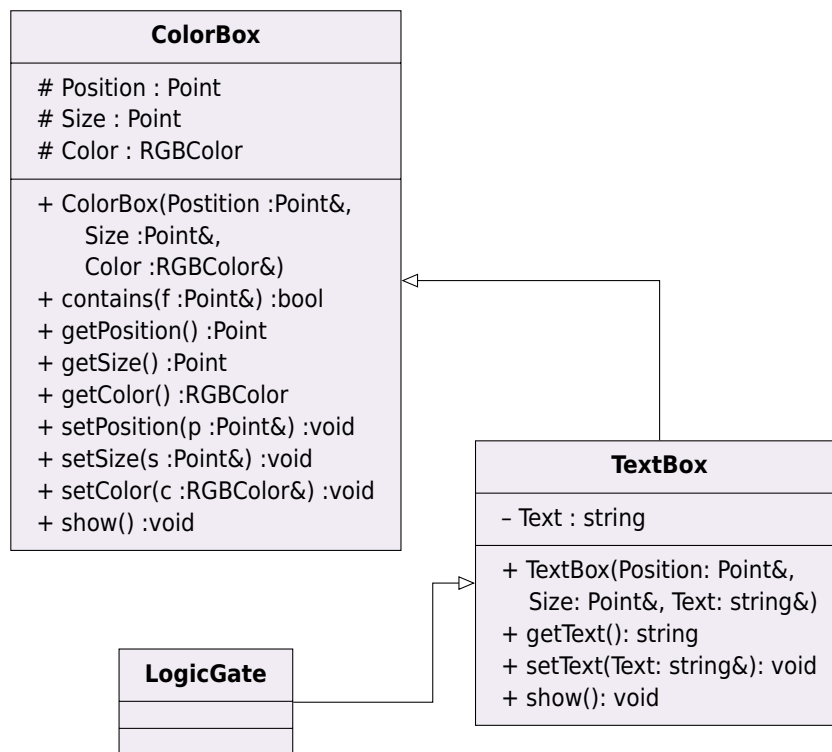


Abbildung 4: Klassendiagramm der Klasse `LogicGate` als Ableitung der Klasse `TextBox`.

3.1 Beschreibung der Datenelemente

Die Klasse `LogicGate` besitzt relativ wenig eigene Datenelemente, da sie einerseits bereits vieles, das sie braucht, erbt und andererseits vieles in die Klassen ihrer Datenelemente ausgelagert ist.

Output: Dieses Objekt ist eine Instanz der (noch zu entwickelnden) Klasse `OutputSignal` (siehe Abschnitt 5), da derzeit nur ein einzelner Ausgang unterstützt wird (an den aber mehrere Eingänge angeschlossen werden können).

Es ist für die Verwaltung des Ausgangszustands ebenso verantwortlich, wie für die Kommunikation des Zustands an die angeschlossenen Eingänge anderer Gatter und das Zeichnen einer Verbindungslinie auf dem Bildschirm. Die Klasse `OutputSignal` ist dafür als Ableitung von `TextBox` gut vorbereitet. Die Klasse `LogicGate` unterstützt nur einen Ausgang¹, der jedoch mit beliebig vielen Eingängen verbunden werden kann.

Input: Dieses Container-Objekt des Typs `std::vector` verwaltet alle `InputSignal`-Objekte (siehe Abschnitt 4) der Eingänge des Gatterbausteins. Es werden in diesem Fall direkt die Objekte im Container gespeichert und keine Pointer, um Arbeit für die (unnötige) dynamische Speicherverwaltung zu sparen.

¹Auch wenn an einigen Stellen Vorbereitungen für eine Erweiterung getroffen wurden.

Dabei wird eine beliebige Anzahl von Eingängen unterstützt, wobei jeder von Ihnen über ein eigenes Objekt der Klasse `InputSignal` abgebildet wird, die dafür alle in dem erwähnten Container namens `Input`, einer Instanz der Klasse `std::vector<InputSignal>` liegen.

Jedes dieser `InputSignal`-Objekte besitzt dabei eine eigene Bildschirmposition, verwaltet die Verbindung zum ggf. verbundenen `OutputSignal` eines anderen Gatters und informiert das eigene `LogicGate`-Objekt im Falle einer Signaländerung.

Indicator: Dieses Objekt ist eine Instanz von `FarbRect` und soll mit seiner Farbe den aktuellen Logikzustand des Ausgangs anzeigen.

ID: Dieser String soll das Bauteil innerhalb der Gesamtschaltung identifizieren. Während der von `TextBox` geerbte Teil zur Angabe der Funktion des Bauteils (z.B. „AND“ für ein Gatter mit AND-Logik) genutzt wird, geht es hier eher um die Identifikation wie z.B. „G42“ für das Gatter mit der Nummer „42“ in der jeweiligen Schaltung. Abbildung 3 zeigt das beabsichtigte Aussehen in einer Vorabskizze.

3.2 Beschreibung der Funktionen

LogicGate: Der Konstruktor verwendet die Parameter `Operation` und `Position` um den Basisinitialisierer mit Parametern zu versorgen. Zusätzlich wird hierfür noch eine Größe von 100x70 Pixeln übergeben. Der Parameter `ID` initialisiert das entsprechende Datenelement. `NumInputs` schließlich wird verwendet, um den Vektor `Input` mit der nötigen Anzahl an `InputSignal`-Objekten zu füllen. Im ausführbaren Teil wird über die Ausführung von `positionElements` die korrekte Positionierung aller Elemente sichergestellt und mit einem Aufruf von `updateIndicator` der Indikator korrekt eingefärbt.

~LogicGate: Der Destruktor ist in dieser Klasse leer.

operator==: Der Vergleichsoperator soll zwei `LogicGate` Objekte vergleichen, indem die `ID` Datenelemente verglichen werden. Zwei `LogicGate`-Objekte sollen also „gleich“ sein, wenn die Werte ihrer `ID`-Elemente gleich sind.

getID: Liefert den String der `ID` als konstante Referenz zurück.

setPosition: Ruft `setPosition` der Basisklasse auf und positioniert dann alle zum Objekt gehörenden `InputSignal`-Objekte und `OutputSignal`-Objekte sowie das Indikatorobjekt passend zur neuen Position des Gatters. Diese zweite Aufgabe erledigt `positionElements`.

setSize: Ruft zuerst die Funktion `setSize` der Basisklasse auf und positioniert dann alle zum Objekt gehörenden Elemente über `positionElements`.

show: Zeichnet zunächst alle `OutputSignal`- und `InputSignal`-Objekte, ruft dann die Funktion `show` der Basisklasse auf, gibt die ID aus, und gibt weitere „Dekorationen“ aus (insbesondere das Indikatorobjekt) durch den Aufruf von `decorate`.

updateOutput: Diese Funktion muss die Eingangssignale entsprechend der vom Gatter abgebildeten Logikfunktion verarbeiten und die Ausgänge entsprechend setzen. Sie wird typischerweise in allen abgeleiteten Klassen neu implementiert.

getOutput: Liefert den aktuellen Ausgabewert².

getNumOutputs: Liefert die Anzahl der Ausgabesignale (derzeit immer 1).

getNumInputs: Liefert die Anzahl der Eingabesignale (wird bei Konstruktion festgelegt).

connectOutput: Diese Funktion ist eine zentrale Benutzerschnittstelle der Klasse und dient dazu, den Ausgang des Gatters mit dem Eingang eines anderen Gatters zu verbinden bzw. die Erstellung dieser Verbindung zu initiieren. Als Parameter ist einerseits das Gatter, mit dessen Eingang die Verbindung hergestellt werden soll, zu übergeben und andererseits die Nummer des Eingangs (beginnend bei Null mit einer Obergrenze je nach Ausstattung des Zielgatters). Die Verbindung wird durch Aufruf der Funktion `connectToConsumer` des `OutputSignal`-Objekts hergestellt, welches für den betreffenden Ausgang zuständig ist, der verbunden werden soll. Zur Illustration siehe auch Abbildung 5.

connectInput: Diese Funktion wird typischerweise von `connectToConsumer`-Funktionen der `OutputSignal`-Objekte anderer Gatter aufgerufen und ermittelt anhand des übermittelten Werts im Parameter `Index` das korrekte `InputSignal`-Objekt. Sofern dieses nicht existiert, weil der übermittelte Wert höher ist als die Anzahl an verfügbaren Eingängen des Gatters, wird zur Rückmeldung dieses Fehlers eine `ExceptionIllegalInputChannel`-Exception³ geworfen. Existiert das `InputSignal`-Objekt jedoch, so wird dessen Funktion `connect` aufgerufen und die beiden per Parameter erhaltenen Werte für das „anfragende“ `OutputSignal`-Objekt sowie den aktuellen Logikwert übergeben. Zur Illustration siehe auch Abbildung 5.

decorate(): Zeichnet das Indikatorobjekt durch Aufruf seiner Methode `show`. Diese Funktion wird typischerweise in vielen abgeleiteten Klassen neu implementiert.

setOutput: Setzt den Ausgang des Gatters auf den als erstem Parameter übergebenen Wert⁴. Hier ist auch ein guter Platz um alle Dinge zu erledigen, die getan werden müssen, wenn der Zustand des Ausgangs sich ändert – insbesondere die Aktualisierung des Indikatorelements (siehe `updateIndicator`) sollte hier durchgeführt werden.

²Der Parameter ist für eine Erweiterung auf mehrere Ausgabesignale vorhanden und wird derzeit nicht genutzt.

³Sie finden alle in dieser Aufgabe verwendeten Exceptions vorimplementiert in `LogicExceptions.h`.

⁴Der zweite Parameter ist für eine Erweiterung auf mehrere Ausgabesignale vorhanden und wird derzeit nicht genutzt.

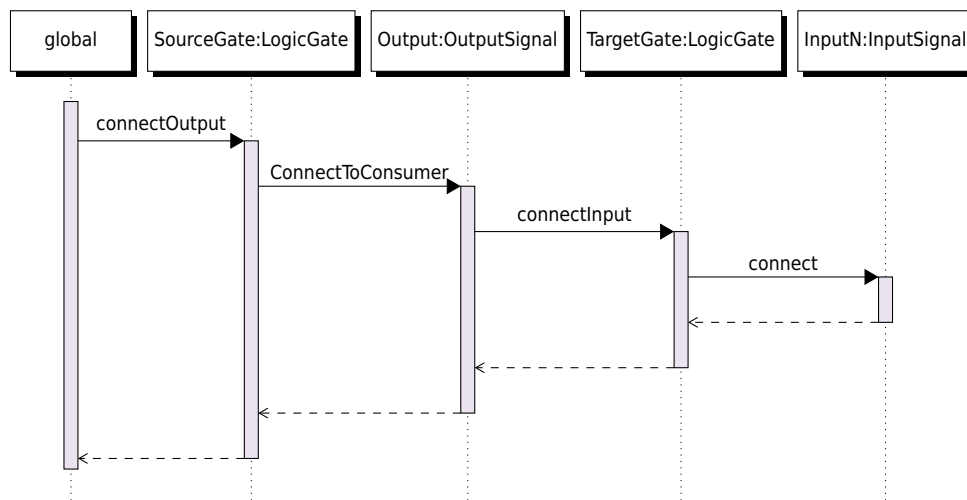


Abbildung 5: Sequenzdiagramm, welches exemplarisch den Aufbau einer Verbindung von einem Gatterausgang „SourceGate“ zu einem Gattereingang „TargetGate“ zeigt.

updateIndicator: Setzt die Farbe des Indikatorobjekts entsprechend des aktuellen Ausgabezustands in **Output** auf rot (false) oder grün (true).

positionElements: Positioniert alle Elemente des Gatters relativ zu seiner aktuellen Position. Das Indikatorobjekt soll in jeder Richtung 15 Pixel von der rechten unteren Ecke des Gatters entfernt und die **InputSignal**-Objekte sollen direkt links vom linken Rand des Gatters platziert werden, die **OutputSignal**-Objekte entsprechend rechts vom rechten Rand des Gatters. Vertikal sollen die Objekte so angeordnet werden, dass sie sich gleichmäßig über die Gatterhöhe zwischen den Eckpunkten des Gatters verteilen. Die Skizze in Abbildung 3 vermittelt einen groben Eindruck.

3.3 Copy-Konstruktor und Zuweisungsoperator

Nachdem die Gatter relativ komplex aufgebaut sind, stellt sich die Frage nach der Notwendigkeit von speziellen Implementierungen von Copy-Konstruktor und Zuweisungsoperator.

Frage: Welche Überlegungen sind anzustellen um zu klären, wie man ein **LogicGate** Objekt korrekt kopieren müsste? Wo liegen die Schwierigkeiten? Diskutieren Sie die Fragestellung mit Ihren Betreuern bei der Abgabe.



Überlegen Sie sich, wie man ein **LogicGate** Objekt kopieren bzw. zuweisen müsste.

Da wir im Moment nicht wissen, wie die Kopie von Statten gehen soll und daher auch nicht wissen, ob die Default-Implementierung der flachen Kopie das gewünschte Ergebnis liefern würde, ist es am besten, das Kopieren und Zuweisen der **LogicGate**-Objekte

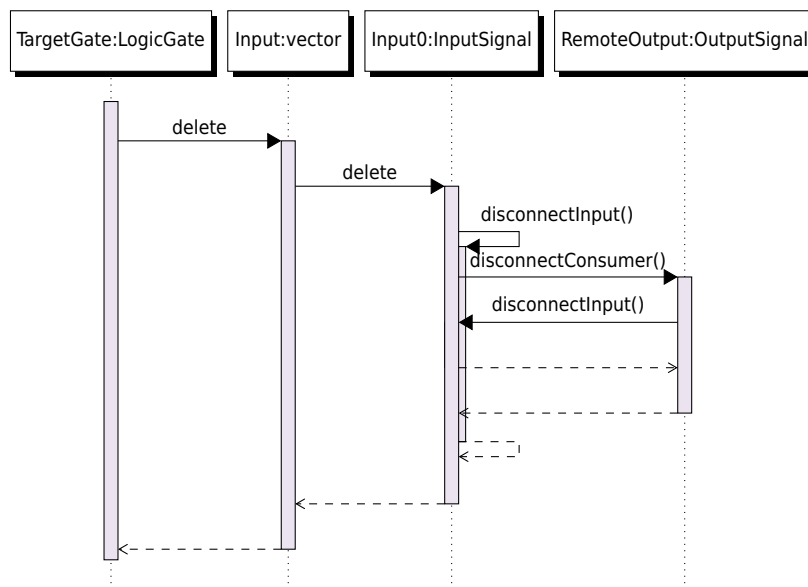


Abbildung 6: Sequenzdiagramm, welches exemplarisch den Abbau einer bestehenden Verbindung zwischen dem Ausgang des `LogicGate`-Objekts `SourceGate` auf den der Pointer `RemoteOutput` des verbundenen `InputSignal`-Objekts des `LogicGate`-Objekts `TargetGate` zeigt, wenn das Objekt `TargetGate` gelöscht wird. Das einzige zu `SourceGate` gehörende Objekt ist dabei das mit `RemoteOutput` bezeichnete.

komplett zu unterbinden. Dafür müssen die beiden Default-Implementierungen gelöscht werden. Dies geschieht in C++ mit der speziellen Syntax

```

class myclass
{
    public:
        myclass(const myclass& orig) = delete;
        myclass& operator=(const myclass& rhs) = delete;
}
  
```

Dadurch produzieren Versuche von Zuweisungen oder der Anfertigung von Kopien der Objekte dieser Klasse `Myclass` Compilerfehlermeldungen.

3.4 Implementierung bei LogicGate

Die `.cpp`-Datei enthält bereits alle Funktionen der Klasse. Allerdings sind die meisten nicht implementiert sondern enthalten nur eine Exception `ExceptionFunctionNotImplemented`, damit es nicht versehentlich zur Ausführung der Funktionen kommt.

Implementieren Sie nun alle Funktionen der Klasse `LogicGate`, welche noch nicht implementiert sind, entsprechend der vorangehenden Beschreibung. Sie können hierfür nach der



Implementieren Sie die Funktionen der Klasse `LogicGate`.

`ExceptionFunctionNotImplemented` suchen um alle nicht implementierten Funktionen in der Datei `LogicGate.cpp` zu finden.

Hinweis: Sie können Ihr Projekt während der Implementierung teilweise compilieren. Allerdings gibt es wegen der starken Verflechtung der Klasse `LogicGate` mit `InputSignal` und `OutputSignal` noch keine Testausgabe.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

4 Implementierung und Integration der Klasse `InputSignal`

InputSignal
– LocalGate :LogicGate& – RemoteOutput OutputSignal* – State :bool – Connected :bool
+ InputSignal(- Localgate :LogicGate& - Position :Point& - InitialState :bool + ~InputSignal() + setState(NewState :bool) :void + disconnectInput() :void + connect(From :OutputSignal& - Initial State :bool): InputSignal& + getState() :bool + isConnected() :bool

Die Klasse `InputSignal` bildet über ihre Instanzen, wie bereits in den vorangehenden Abschnitten angedeutet, die Eingangssignale eines Logikgatters funktional aber auch bezüglich der Bildschirmdarstellung ab. Die von ihr gebildeten Objekte verwalten den Verbindungszustand, den Kommunikationspartner, die Bildschirmposition und den aktuellen Signalwert.

Die Klasse wird durch eine Ableitung von `ColorBox` erzeugt, wodurch die erforderliche Speicherung der Bildschirmkoordinaten und die Darstellung auf dem Bildschirm bereits funktional abgedeckt sind.

Damit bleiben nur noch die übrigen Aufgaben zu implementieren.

4.1 Beschreibung der Datenelemente

LocalGate: Diese **Referenz** auf ein Objekt der Klasse `LogicGate` dient dazu, mit dem Gatter zu kommunizieren, zu dem das `InputSignal`-Objekt gehört. Diese Information ist insbesondere nötig, um das Gatter über Änderungen beim Eingangssignal zu informieren.

Da es sich bei diesem Datenelement um eine Referenz handelt, kann es nur einmalig in der Initialisiererliste des Konstruktors mit einem Wert versehen werden. Das ist aber gewünscht, da die Eingänge ja fest mit „ihrem“ Gatter verbunden sind.

RemoteOutput: Dieser **Pointer** auf ein ggf. verbundenes **OutputSignal** eines anderen Gatters wird normalerweise nicht benötigt. Einziger Anwendungsfall ist der Destruktor, der es notwendig macht, dass sich ein **InputSignal** bei seinem verbundenen **OutputSignal** abmeldet.

State: Vom Typ **bool** speichert dieses Element den zuletzt vom verbundenen **OutputSignal** übermittelten Logikwert (oder **false** falls keine Verbindung besteht).

Connected: Ebenfalls vom Typ **bool** verwaltet diese Variable den Verbindungsstatus.

4.2 Beschreibung der Funktionen

InputSignal: Die Parameter werden verwendet, um die Datenelemente des Objekts mit den entsprechenden Werten vorzubelegen.

~InputSignal: Bevor das **InputSignal**-Objekt zerstört wird, muss es hier im Konstruktor noch dafür sorgen, bei einem eventuell verbundenen **OutputSignal** abgemeldet zu werden, damit dieses nicht versucht, mit dem nicht mehr existenten **InputSignal** zu kommunizieren. Für diesen Abmeldevorgang gibt es die Funktion **disconnectInput**.

setState: Diese Schnittstelle erlaubt es einem verbundenen **OutputSignal**-Objekt, neue Werte an das **InputSignal**-Objekt zu kommunizieren.

Der als **bool**-Parameter übergebene neue Logikwert wird in **State** abgelegt und das eigene lokale Gatter angewiesen, die Ausgänge neu zu berechnen, indem die Funktion **updateOutput** des Gatters aufgerufen wird.

disconnectInput: Diese Funktion meldet das Objekt beim verbundenen **OutputSignal** korrekt ab. Danach ist der Input wieder frei für neue Verbindungen. Für dieses Abmelden gibt es beim **OutputSignal** die Methode **disconnectConsumer**, der die eigene Identität als Parameter zu übergeben ist. Zudem ist noch die „interne Buchhaltung“ zu bereinigen, indem der Verbindungsstatus zurückgesetzt und der Pointer auf das verbundene **OutputSignal** mit dem **nullptr** gefüllt wird.

Wichtig: Um später Endlosschleifen zu vermeiden, ist es unerlässlich, dass hier zunächst geprüft wird, ob das Objekt überhaupt verbunden ist. Nur falls dies der Fall ist, sind überhaupt Aktivitäten nötig.

Hinweis: Siehe dazu auch Abbildung 6

connect: Falls das `InputSignal` bereits verbunden ist, wird eine Exception des Typs `ExceptionInputConnected` ausgelöst.

Ansonsten wird:

- das übergebene `OutputSignal`-Objekt als Kommunikationspartner gespeichert,
- das `InputSignal`-Objekt als verbunden gekennzeichnet,
- der initiale Logikwert aus der Parameterliste als Wert übernommen,
- das eigene Gatter zur Neuberechnung der Ausgänge aufgefordert,
- und schließlich die eigene Identität als Rückgabewert zurückgeliefert.

getState: Liefert den Wert des aktuellen Logikzustands zurück.

isConnected: Gestattet den Lesezugriff auf den in `isConnected` gespeicherten Verbindungsstatus.

4.3 Implementierung bei InputSignal

Die `.cpp`-Datei enthält bereits alle Funktionen der Klasse. Allerdings sind die meisten inhaltlich nicht implementiert, sondern enthalten nur eine Exception `ExceptionFunctionNotImplemented`, damit es nicht unbemerkt zur Ausführung der Funktionen kommt.

Implementieren Sie nun alle Funktionen der Klasse `InputSignal`, welche noch nicht implementiert sind, entsprechend der vorangehenden Beschreibungen. Sie können hierfür nach der Verwendung von `ExceptionFunctionNotImplemented` suchen, um alle nicht implementierten Funktionen in der Datei `InputSignal.cpp` zu finden.

Hinweis: Sie können Ihr Projekt während der Implementierung testweise mit der Einstellung `TEST_LEVEL TEST_INPUTSIGNAL` compilieren. Allerdings gibt es wegen der starken Verflechtung der Klasse `LogicGate`, `InputSignal` und `OutputSignal` noch keine Testausgabe.



Implementieren Sie die Funktionen der Klasse `InputSignal`.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

5 Implementierung und Integration der Klasse OutputSignal

Die Objekte der Klasse `OutputSignal` verwalten die Verbindungen der Ausgänge zu den Eingängen. Dafür pflegen sie eine Liste mit verbundenen

InputSignal-Objekten. Sie sind auch dafür zuständig, diese Objekte über Änderungen des Signalpegels zu informieren.

Schließlich kümmern sich die **OutputSignal**-Objekte auch um die graphische Ausgabe der Verbindungsleitungen zwischen dem Ausgang und dem Eingang.

Dafür benötigen die **OutputSignal**-Objekte eine Bildschirmposition und sind daher von **ColorBox** abgeleitet, was auch die Implementierung ihrer eigenen Darstellung auf dem Bildschirm erledigt.

5.1 Beschreibung der Datenelemente

FanOut: Dieser `std::vector` speichert Pointer auf alle mit dem Ausgang verbundenen **InputSignal**-Objekte.

LastState: Hier wird der zuletzt an die in **FanOut** gespeicherten verbundenen **InputSignal**-Objekte kommunizierte Logikwert abgelegt. Dies wird insbesondere dafür genutzt, unnötige Doppelungen bei der Übertragung der Werte an diese Objekte zu verhindern.

5.2 Beschreibung der Funktionen

OutputSignal: Der Konstruktor nutzt die per Parameter übergebene Position um sie dem Basisinitialisierer als Position zu übergeben. Als Abmessung wird eine Größe von 2x4 Pixeln und als Füllfarbe ein mittleres Grau verwendet. Der zweite Parameter wird benutzt um den „alten“ Zustand des Ausgangs in **LastState** zu initialisieren.

~OutputSignal: Der Destruktor muss die Verbindung zu allen verbundenen **InputSignal**-Objekten abbauen.

sendState: Falls der als Parameter übergebene neue Ausgangszustand ungleich dem gespeicherten zuletzt gesendeten Zustand ist, wird der neue Zustand an alle verbundenen **InputSignal**-Objekte gesendet, indem jeweils deren **setState**-Funktionen aufgerufen werden. Zudem wird der neue Zustand dann als zuletzt gesendeter Zustand gespeichert.

getLastState: Erlaubt das Auslesen des aktuellen (also des zuletzt gesendeten) Zustands des Ausgangs.

show: Zeichnet Verbindungslinien zu allen derzeit verbundenen **InputSignal**-Objekten. Im Anschluss wird der **OutputSignal**-Anschluss selbst über den Aufruf von **show** der Basisklasse gezeichnet.

connectToConsumer: Diese Funktion wird vom `LogicGate`, zu dem das `OutputSignal` gehört, aufgerufen. Sie dient dazu, die Verbindung zu einem `InputSignal` eines anderen `LogicGate`-Objekts herzustellen. Hierfür wird die Funktion `connectInput` des anderen Gatters, welches als Parameter übergeben wurde, aufgerufen und an diese die Nummer des gewünschten Anschlusses (die ebenfalls als Parameter übergeben wurde) sowie der aktuelle Logikwert des Ausgangssignals übermittelt. Ein Pointer auf das von dieser Funktion zurückgelieferte `InputSignal`-Objekt wird dann an den `std::vector FanOut` angehängt.

disconnectConsumer: Diese Funktion dient in erster Linie dazu, die Verbindung zu einem `InputSignal` abzubauen, wenn dieses die Trennung anfordert (z.B. in seinem Destruktor). Dafür übermittelt dieses Objekt, das die Trennung anfordert, seine eigene Identität als Parameter. Zunächst wird daraufhin die Funktion `disconnectInput` des anfordernden Objekts aufgerufen, dann in der Liste der verbundenen Objekte in `FanOut` nach dem passenden Pointer gesucht und schließlich dieser Eintrag aus dem `std::vector`-Objekt gelöscht.

Hinweis: Siehe dazu auch Abbildung 6

5.3 Implementierung bei OutputSignal

Die `.cpp`-Datei enthält bereits alle Funktionen der Klasse. Allerdings sind die meisten inhaltlich nicht implementiert, sondern enthalten nur eine Exception `ExceptionFunctionNotImplemented`, damit es nicht unbemerkt zur Ausführung der Funktionen kommt.

Implementieren Sie nun alle Funktionen der Klasse `OutputSignal`, welche noch nicht implementiert sind, entsprechend der vorangehenden Beschreibungen. Sie können hierfür nach der Verwendung von `ExceptionFunctionNotImplemented` suchen, um alle nicht implementierten Funktionen in der Datei `InputSignal.cpp` zu finden.

Hinweis: Sie können Ihr Projekt während der Implementierung teilweise mit der Einstellung `TEST_LEVEL TEST_OUTPUTSIGNAL` compilieren. Allerdings gibt es wegen der starken Verflechtung der Klasse `LogicGate`, `InputSignal` und `OutputSignal` noch keine Testausgabe.



Implementieren Sie die Funktionen der Klasse `OutputSignal`.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

6 Ein erster richtiger Test

Testen Sie Ihre Implementierung, indem Sie Ihr Programm mit dem `TEST_LEVEL TEST_LOGICGATECOMM` übersetzen und ausführen.

Das Ergebnis sollte aussehen wie in Abbildung 7 dargestellt.

Analysieren Sie den durch den aktuellen Testschalterwert in `main.cpp` aktivierten Code, insbesondere in `VtlInit`. Dieser löst den in Abbildung 5 dargestellten Ablauf aus.

Die `LogicGate`-Implementierung ist damit im Prinzip fertig. Etwas unschön ist es, dass `LogicGate` noch keinerlei Funktion besitzt, d.h. es kann den Zustand des Ausgangs nicht aus den Eingangszuständen berechnen.

Dies ist ein typisches Beispiel für den Fall, dass eine Basisklasse erzeugt wird, die alle Gemeinsamkeiten einer ganzen Familie von Klassen enthält, aber der die von den abgeleiteten Klassen gelieferten spezifischen Funktionen fehlen. Diese Klassen lassen sich nicht sinnvoll zur Instanziierung neuer Objekte nutzen, da ihnen dann eine wichtige Funktionalität fehlt. Daher ist es keine gute Idee, solche Funktionen mit einer Dummy-Implementierung zu versehen (hier in diesem Fall eine leere Funktion).

Um zu vermeiden, dass ein derart nicht funktionsfähiges Objekt der Klasse `LogicGate` versehentlich in eine Logikschaltung integriert wird, ändern Sie die Definition der Klasse so ab, dass die Funktion `updateOutput` eine „pure function“ wird.

Wiederholen Sie den Compilerlauf und analysieren Sie die resultierende Fehlermeldung. Versuchen Sie die Struktur der Fehlermeldung zu verstehen um sie bei Ihrem nächsten „Aufeinandertreffen“ interpretieren zu können.



Erzeugen Sie nun eine neue Revision und laden Sie den aktuellen Stand auf den Kursserver hoch.



Testen Sie Ihr Programm mit dem `TEST_LEVEL TEST_LOGICGATECOMM`.

Recherche ☕☕

„rein virtuelle“ Funktionen und abstrakte Klassen



Erzeugen Sie eine „pure function“.

7 Ableitung der speziellen Gatter

Da Logikschaltungen mit abstrakten Logikgattern wenig Nutzen bringen würden (und jetzt auch nicht mehr möglich sind), sollen nun entsprechend spezialisierte Gatter für unterschiedliche Logikoperationen abgeleitet werden.

Jemand aus Ihrem Entwicklungsteam hat bereits vorgelegt und eine Klasse für ein `NOT`-Gatter erzeugt. Sie finden diese in den Dateien `SimpleGates.h/.cpp`. Sie erkennen, dass nur sehr wenige Funktionen implementiert werden müssen. Versuchen Sie zu verstehen, wie diese mit den Funktionen in `LogicGate` zusammenarbeiten.



Erzeugen Sie spezialisierte Ableitungen der Klasse `LogicGate` für die Logikoperationen AND, OR, XOR und NAND.



Analysieren Sie den Programmcode der Klasse `LogicGateNOT`.

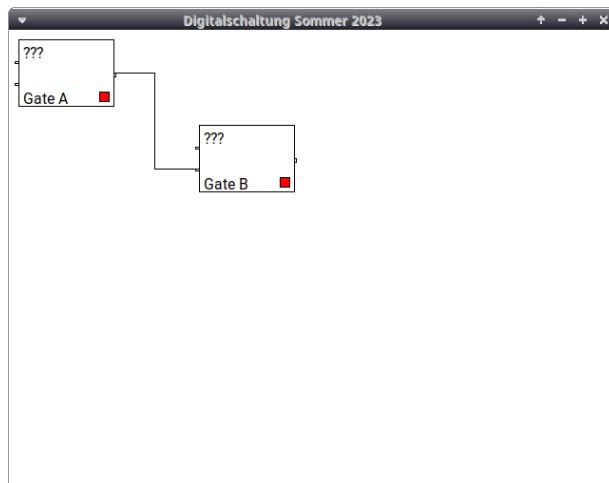


Abbildung 7: Ein Erster Test der Klasse `LogicGate`. Die Darstellung ist weitgehend funktionsfähig. Allerdings haben die Gatter noch keinerlei Funktion, welche den Output aus den Inputs berechnen würde. Dies ist ein Beispiel für eine Klasse, die abstrakt sein sollte, weil sie ihre Funktion noch nicht erfüllen kann, sondern erst die Ableitungen dies können werden.

Frage: Warum besitzt `LogicGateNOT` keine eigene Implementierung von `show`? Warum kann die eine existierende Implementierung von `LogicGate::show` nun plötzlich unterschiedlich aussehende Gatter darstellen?

Erzeugen Sie nun neue Klassen in `SimpleGates.h/.cpp` als Ableitung von der Klasse `LogicGate`, welche jeweils eine der Operationen AND, OR, XOR und NAND und eine variable Anzahl an Eingängen unterstützen. Benennen Sie die Klassen mit `LogicGateAND`, `LogicGateOR`, `LogicGateXOR` und `LogicGateNAND`.

Sorgen Sie dafür, dass die jeweiligen Klassen durch entsprechende **Dekoration** so unterschiedlich aussehen, wie dies in Abbildung 1 zu erkennen ist:

- Das NAND-Gatter soll am Ausgang des Gatters einen kleinen schwarzen Kreis aufweisen, der die Invertierung des Ausgangs andeutet.
- Das XOR-Gatter soll an der Ausgangsseite des Gatters einen schwarzen Balken erhalten.

Die Klassen sollen jeweils den Namen der unterstützten Logikoperation als Benennung (links oben) anzeigen und die Konstruktoren der neuen Klassen sollen daher auch keinen Parameter zur Übergabe des Gatternamens mehr besitzen, nur die **ID** (Defaultwert: Leerstring) und die Anzahl der Eingänge (Defaultwert: zwei) sollen noch zusätzlich zur Position übergeben werden können.

Frage: Welche dieser Klassen benötigen eine eigene Implementierung der Funktion `decorate`? Warum?



Testen Sie Ihr Programm mit dem `TEST_LEVEL` `TEST_SIMPLEGATES`.

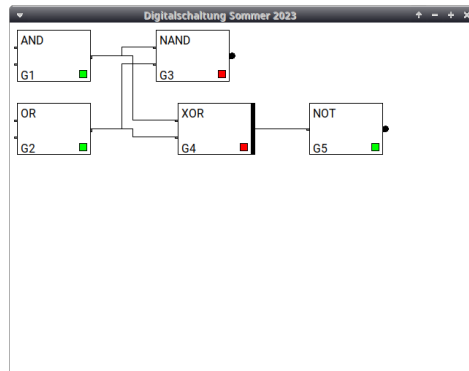


Abbildung 8: Erster Test der spezialisierten Gatter.

Kompilieren und testen Sie Ihr Programm mit dem `TEST_LEVEL` `TEST_SIMPLEGATES` und beheben Sie ggf. auftretende Probleme. Die Ausgabe sollte ähnlich zu Abbildung 8 dargestellt werden.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

8 Schalter zur Signaleingabe

LogicGateSwitch
SwitchElement :Switch* - TimedSwitchElement :SwitchPulse*
+ LogicGateSwitch(Label :string&, Position :Point&, ID :string& Type :unsigned, Timing :int) + ~LogicGateSwitch() + updateOutput() :bool + onMouse(where :Point&) :void + decorate() :void + clock() :void

Damit die Gatter manuell dynamisch mit variablen Eingangssignalen beschaltet werden können, sollen nun entsprechende Dialogelemente basierend auf der Klasse `LogicGate` entwickelt werden, welche Schalter aus der vorangehenden Übungsaufgabe als Bedienelemente enthalten.

Leiten Sie unter Verwendung der bereits vorhandenen Dateien `LogicGateSwitch.h/.cpp` eine neue Klasse `LogicGateSwitch` von der Klasse `LogicGate` ab. Nebenstehend sind die erforderlichen Methoden dargestellt. Die Klasse soll ein Schalterelement verwalten und dessen Zustand an den von `LogicGate` geerbten Output weitergeben, so dass sich

der Schalter in der Digitalschaltung verhält wie ein Logikgatter ohne Eingang.

Im Einzelnen sollten Sie bei der Implementierung folgendermaßen vorgehen:

Konstruktor: Hier verwenden Sie den Basisinitialisierer in der Initialisiererliste, um den Konstruktor von `LogicGate` zu nutzen. Dabei



Erstellen Sie eine neue Klasse `LogicGateSwitch` welche die Eingabe von Logikpegeln in die Logikschaltungen erlaubt.

verwenden Sie den Wert des Parameters `Label` des Konstruktors als Wert des Parameters `Operation` des Basisinitialisierers und reichen die `Position` und die `ID` durch.

Zusätzlich muss je nach Wert des Parameters `Type` (Der Defaultwert soll 0 sein.) ein `SwitchOnOff`- (`Type` ist 0), `SwitchToggle`- (1) oder `SwitchPulse`-Objekt (2) dynamisch erzeugt und der Pointer darauf in `SwitchElement` abgelegt werden. Hierfür verwenden Sie am Besten den ausführbaren Teil des Konstruktors.

Im Falle eines `SwitchPulse`-Objekts als Schalterelement wird der Pointer auf das Objekt zusätzlich in `TimedSwitchElement` abgelegt, welches in allen anderen Fällen den Wert `nullptr` besitzen soll.

Das Argument `Timing` soll den Defaultwert 100 besitzen und wird bei Bedienelementen mit Zeitverhalten (derzeit nur `SwitchPule`-Objekte deren Konstruktor übergeben um deren Zeitkonstante festzulegen. Bei anderen Bedienelementen hat dieser Parameter keine Funktion.

Die Position des Schalters innerhalb des Gatterrahmens soll gegenüber der Position des Gatters ca. 70 Punkte nach rechts und drei Punkte nach unten verschoben werden. Setzen Sie die Abmessungen des `LogicGateSwitch`-Objekts so, dass es 6 Punkte höher und 73 Punkte breiter als das eingebettete `Switch`-Objekt ist.

Hinweis: Verwenden Sie hierfür die vom `Switch`-Objekt gemeldete Größe!

Copykonstruktor und Zuweisungsoperator: Den vorhandenen Default-Copy-Konstruktor bzw. -Zuweisungsoperator löschen Sie jeweils in bewährter Art und Weise.

Destruktor: Hier sind keine zusätzlichen Hinweise notwendig. Sie wissen, was zu tun ist.

onMouse: Ruft die entsprechende Methode des von `SwitchElement` referenzierten `Switch`-Objekts auf und setzt anschließend den Schaltzustand des eingebetteten `Swicth`-Objekts als neuen Zustand des Gatterausgangs.

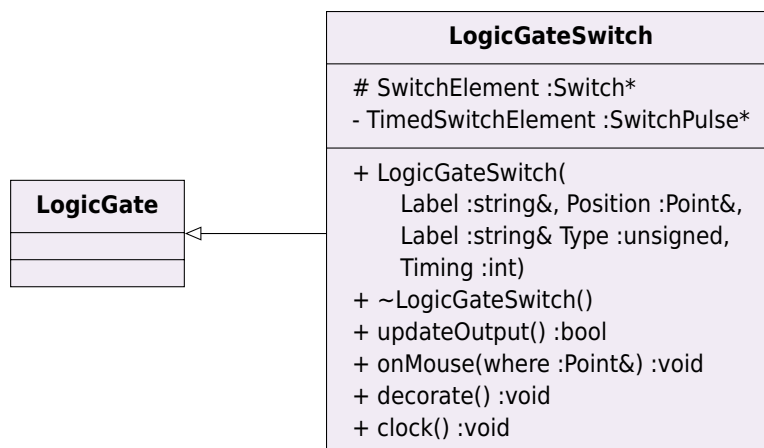


Abbildung 9: Klassendiagramm der Klasse `LogicGateSwitch`.

decorate: Damit das eingebettete `Switch`-Objekt auch dargestellt wird, müssen Sie hier die `show`-Methode des von `SwitchElement` referenzierten `Switch`-Objekts aufrufen.

updateOutput: Diese Funktion wird als leere Funktion implementiert.

clock: Diese Funktion ruft die Methode `onTimerTick` des von `TimedSwitchElement` referenzierten Objekts auf.

Kompilieren und testen Sie Ihr Programm mit dem `TEST_LEVEL TEST_GATESWITCH` und beheben Sie ggf. auftretende Probleme. Die Ausgabe sollte ähnlich zu Abbildung 10 dargestellt werden. Prüfen Sie insbesondere die korrekte Funktion des `SwitchPulse`-Objekts.



Testen Sie Ihr Programm mit dem `TEST_LEVEL TEST_GATESWITCH`.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

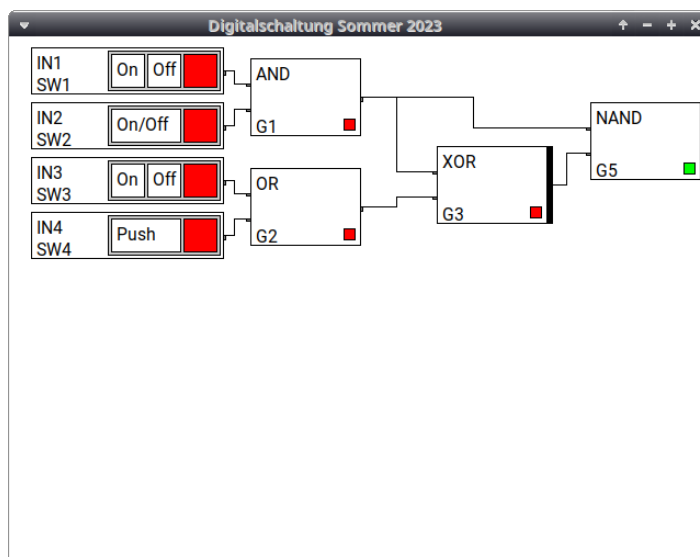


Abbildung 10: Beispiel für eine komplette Digitalschaltung, realisiert mit der von Ihnen entwickelten Simulationsumgebung.

9 Ein Container für die Schaltung

Circuit
– CrystalFrequency :double – Gates :vector<LogicGate*> – Switches :vector<LogicGateSwitch*>
+ Circuit(CrystalFrequency :double) + ~Circuit() + addGate(NewGate :LogicGate*) :void + addSwitch(NewSwitch :LogicGateSwitch*) :void + onMouse(where :Point&) :void + clock() :void + show() :void + operator[](Index :unsigned) :LogicGate& + operator[](ID :string&) :LogicGateSwitch&

Nun ist es an der Zeit, einen Test mit einer umfangreicheren Digitalschaltung anzugehen. Wenn Sie sich den bisherigen Testcode ansehen, dann ist das alles sehr umständlich.

Statt viele Einzelvariablen zu verwenden, würde man gerne Array-artige Datenstrukturen verwenden um mit Schleifen arbeiten zu können. Aber auch dann bleibt die Einrichtung der Verbindungen umständlich.

Praktisch wäre es, die Objekte anhand ihrer **ID** verknüpfen zu können, was eine am Schaltplan orientierte Arbeitsweise erlauben und auch z.B. ein Einlesen der Schaltungsinformation aus einer Datei erleichtern würde.

Daher soll nun eine spezielle Containerklasse entwickelt werden, die ein effizientes Aufbauen von Schaltungen ermöglicht. Ihre Aufgaben sind:

- Verwalten der Objekte
- Verteilen der Aufrufe der **show**-, **clock** und **onMouse**-Funktionen
- Unterstützung des Verbindungsaufbaus durch **ID**-basiertes Finden der Bausteine.

Realisiert wird das durch **std::vector**-Objekte und einige Methoden, welche den Umgang damit erleichtern. Spezielle Iteratorobjekte werden nicht benötigt, da die Klasse **std::vector** alle benötigten Dinge bereits mitbringt.

Die **LogicGateSwitch**-Objekte zeichnen sich durch das Alleinstellungsmerkmal aus, eine Mausbedienung und eine Zeitabhängigkeit zu erfordern bzw. zu unterstützen, weshalb diese speziell behandelt und daher zusätzlich in einer separaten Liste verwaltet werden müssen.

9.1 Beschreibung der Datenelemente

Gates: In diesem Objekt der Klasse **std::vector** werden Pointer auf *alle* Objekte der Digitalschaltung, die von **LogicGate** abstammen, verwaltet.

Switches: In diesem Objekt der Klasse `std::vector` werden Pointer auf die in der Digitalschaltung eingesetzten Objekte des Typs `LogicGateSwitch` verwaltet. Diese sind zusätzlich auch in `Gates` enthalten.

CrystalFrequency: Frequenz des Quarzes, die die Schaltung mit Systemtakt versorgt (für spätere Erweiterungen).

9.2 Beschreibung der Funktionen

Konstruktor: Initialisiert `Gates` und `Switches` mit einer Länge von null Elementen.

Destruktor: Löscht alle von den Pointern in `Gates` referenzierten Objekte.

addGate: Hängt den übergebenen Pointer auf das neue Objekt an den Vektor `Gates` an.

addSwitch: Hängt den übergebenen Pointer auf das neue Objekt an die beiden Vektoren `Gates` und `Switches` an.

clock: Wird entsprechend der Frequenz des Quarzkristalls von außen aufgerufen und ruft seinerseits die `clock`-Methoden der durch die Pointer in `Switches` referenzierten Objekte auf.

show: Ruft die Funktion `show` bei allen von den Pointern in `Gates` referenzierten Objekten auf.

onMouse: Ruft die Funktion `onMouse` bei allen von den Pointern in `Switches` referenzierten Objekten auf.

operator[unsigned]: Gibt eine Referenz auf das Objekt zurück, auf das der Pointer in dem Eintrag in `Gates` mit dem angeforderten Index verweist.

operator[string]: Sucht unter allen Objekten, die von den Pointern in `Gates` referenzierten Objekten eines, das die selbe `ID` besitzt, wie der Parameterwert angibt, und gibt eine Referenz darauf zurück.

Wird kein Objekt mit einer passenden ID gefunden, so wird eine Exception des Typs `ExceptionUnknownLogicGateID` ausgelöst.

9.3 Implementierung von Circuit

Verwenden Sie die beiden bereits existierenden Dateien `Circuit.h/.cpp` um die Klasse `Circuit` zu definieren und ihre Funktionen zu implementieren.

Damit ist es nun möglich, komfortabel auch größere Digitalschaltungen zu erstellen. Analysieren Sie den mit `TEST_LEVEL`



Definieren und implementieren Sie die Klasse `Circuit`.

`TEST_CIRCUIT_AVAILABLE` aktivierten Testcode in `main.cpp` um zu verstehen, wie die neuen Funktionen von `Circuit` genutzt werden können.

Kompilieren und testen Sie dann Ihr Projekt mit der Einstellung `TEST_LEVEL TEST_CIRCUIT_AVAILABLE`.



Testen Sie Ihr Projekt mit `TEST_LEVEL TEST_CIRCUIT_AVAILABLE`.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

10 Eine Digitalschaltung

Konstruieren Sie eine umfangreichere Digitalschaltung bestehend aus ca. 20 Elementen mit Ihrer Simulationsumgebung und testen Sie diese.

Wichtig: Unser Design der Simulationsumgebung erlaubt **keine Signalschleifen**, das heißt, ein Gatter darf weder direkt noch indirekt (also mit anderen zwischengeschalteten Gattern) seinen eigenen Ausgang als Eingang konsumieren. Beachten Sie dies bei der Erstellung Ihrer Digitalschaltung.

Entwerfen Sie nun zur Erklärung des Sachverhalts (als Zeichnung, Code ist nicht nötig) eine Digitalschaltung mit drei Gattern, die so verschaltet sind, dass das „erste“ Gatter seinen eigenen Ausgangszustand – nachdem er die beiden anderen Gatter passiert hat und entsprechend modifiziert wurde – wieder auf einen seiner Eingänge zurückgeführt bekommt. Zeichnen Sie ein Sequenzdiagramm, das zeigt, welche Abfolge von Aktivitäten ausgelöst wird, wenn von diesem ersten Gatter die Methode `updateOutput` aufgerufen wird. Was ist das auftretende Problem?

Frage: Warum sind Signalschleifen ein Problem? Was würde passieren? Was könnte man dagegen unternehmen?



Testen Sie Ihre Klassenfamilie mit einer umfangreichen Digitalschaltung.



Untersuchen Sie mit einem Sequenzdiagramm die Folge von eventuellen Signalschleifen in der Schaltung.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.